TALLER PRACTICO DE SEGUNDO CORTE - DOCKER

Arquitectura de Software · Semana 10 · Entrega individual

Estudiante:

Elías Daniel Ramos Herrera

Código:

T00082523

Profesor:

Luis Giovanny Carreno Ortiz

Universidad Tecnologica de Bolivar

Colombia, Cartagena de indias

13/10/2026

Introducción

Este informe documenta la ejecución práctica de la guía proporcionada en el repositorio biblioteca-api. El objetivo de esta práctica es levantar un entorno completo de desarrollo (API en Go y base de datos MySQL) utilizando Docker y Docker Compose, demostrando la correcta configuración y comunicación entre contenedores.

A continuación, se detalla el paso a paso seguido, acompañado de las evidencias gráficas correspondientes, y se resuelven las preguntas de comprensión técnica.

Fase 1 — Instalación y Verificación

El primer paso fue verificar que el entorno de Docker estuviera correctamente instalado y funcionando en la máquina, junto con el subsistema de Windows para Linux (WSL).

Evidencia: Versiones de Docker
Se ejecutaron los comandos docker --version y docker compose version para confirmar que el motor de Docker y el plugin de Compose estuvieran listos.



Fase 2 — Puesta en Marcha

Una vez verificado el entorno, se procedió a configurar y levantar los servicios definidos en el archivo docker-compose.yml.

Creación del archivo de configuración
Se navegó al directorio del proyecto y se creó el archivo de variables de entorno a partir de la plantilla:

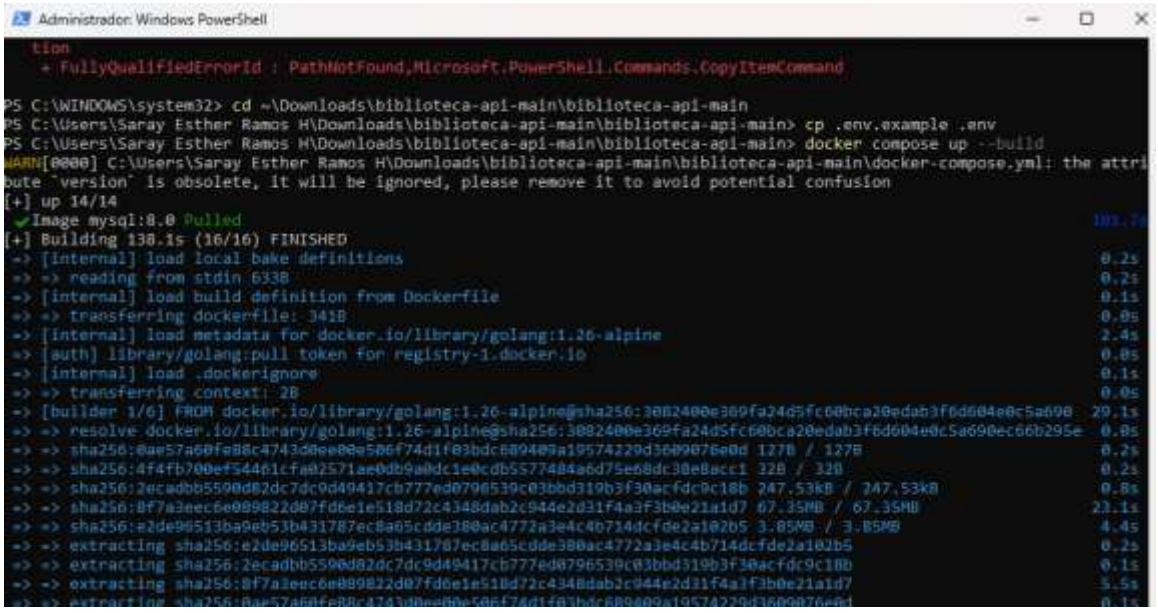
cp .env.example .env

Variable / Valor	Explicación
`DB_HOST=mysql`	Define el nombre del host al que se conectará la API. En este caso, mysql es el nombre del servicio definido en Docker Compose, por lo que Docker resuelve este nombre a la IP interna del contenedor de la base de datos.
`DB_PORT=3306`	El puerto interno en el que MySQL está escuchando dentro de su contenedor.
`DB_USER=biblioteca_user` / `DB_PASSWORD=biblioteca_pass`	Credenciales de acceso creadas automáticamente por la imagen de MySQL al inicializarse.
`DB_NAME=biblioteca`	Nombre de la base de datos que la API utilizará.
`JWT_SECRET=TOKEN_BIBLIOTECA`	La clave criptográfica utilizada para firmar y verificar los tokens de autenticación.
`JWT_EXPIRATION_HOURS=24`	El tiempo de validez de los tokens emitidos.
`APP_PORT=8080`	El puerto en el que la API de Go escuchará las peticiones entrantes.

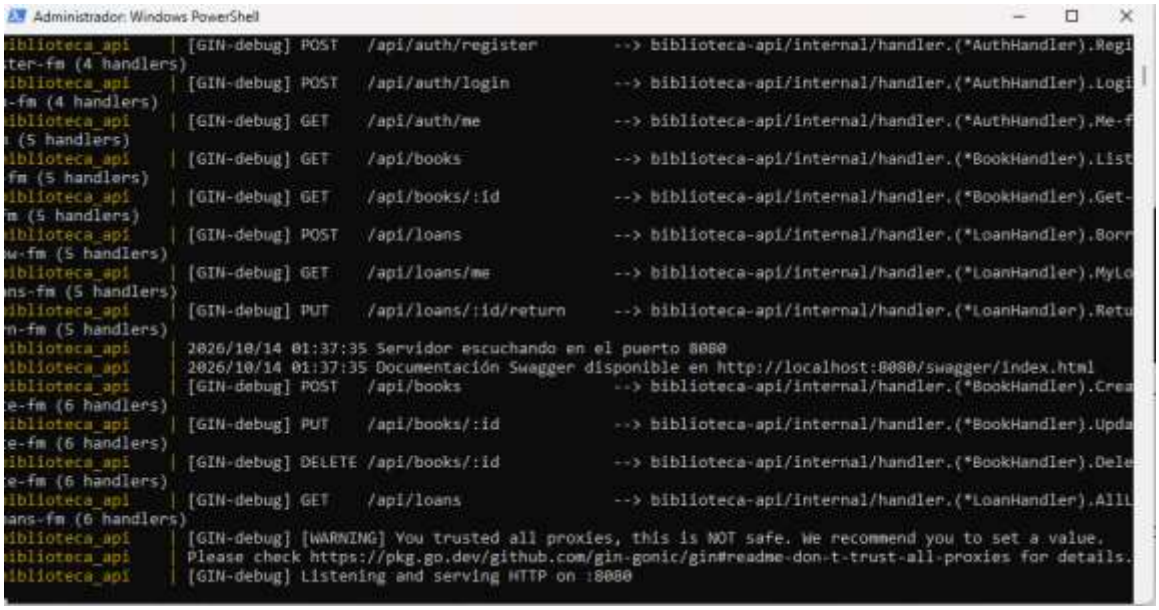
Construcción y Levantamiento de los Servicios
Se ejecutó el comando para descargar las imágenes, compilar la API y levantar los contenedores:

docker compose up -build

- Evidencia: Inicio de la construcción (descargando imágenes y compilando)



- Evidencia: Servicios corriendo exitosamente



Como se observa en la consola, la API se conectó exitosamente a MySQL y el servidor HTTP quedó escuchando en el puerto 8080.

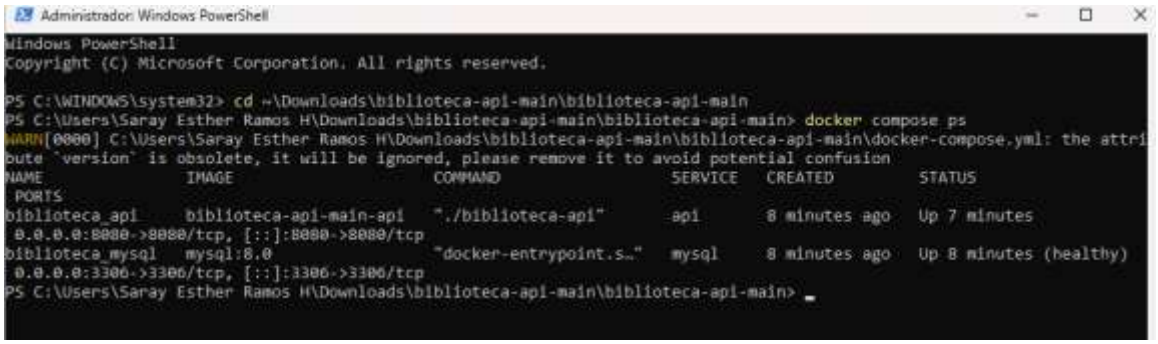
Fase 3 — Verificación

Para confirmar que los contenedores estaban operando correctamente en segundo plano y que la aplicación era accesible, se realizaron las siguientes pruebas.

Estado de los contenedores

En una nueva terminal, se verificó el estado de los servicios con docker compose ps:

- Evidencia: Contenedores en estado 'Up' y MySQL 'healthy'

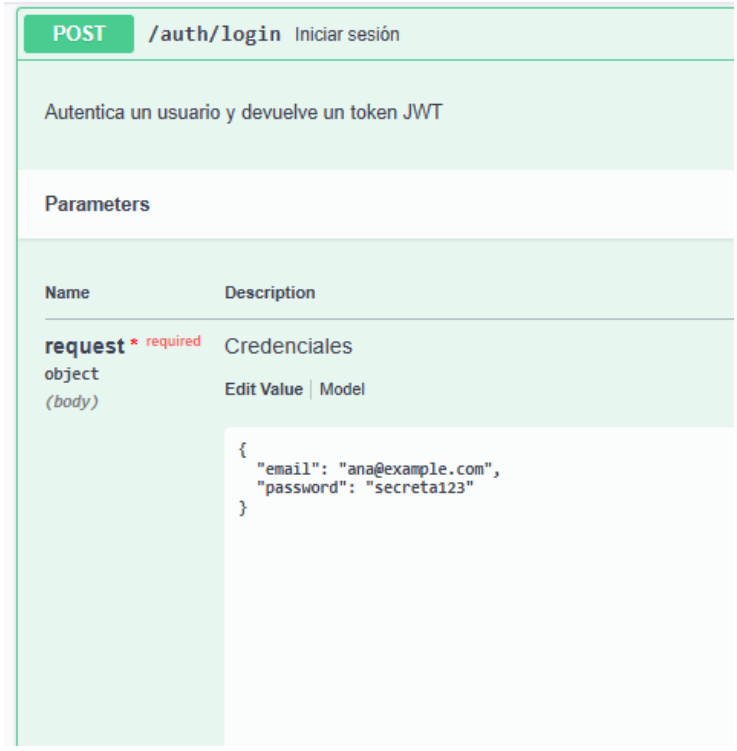


Pruebas de la API (Swagger)

Se accedió a la documentación interactiva en http://localhost:8080/swagger/index.html para probar el flujo completo: registro, inicio de sesión y consulta protegida.

1. Registro de Usuario (POST /auth/register)

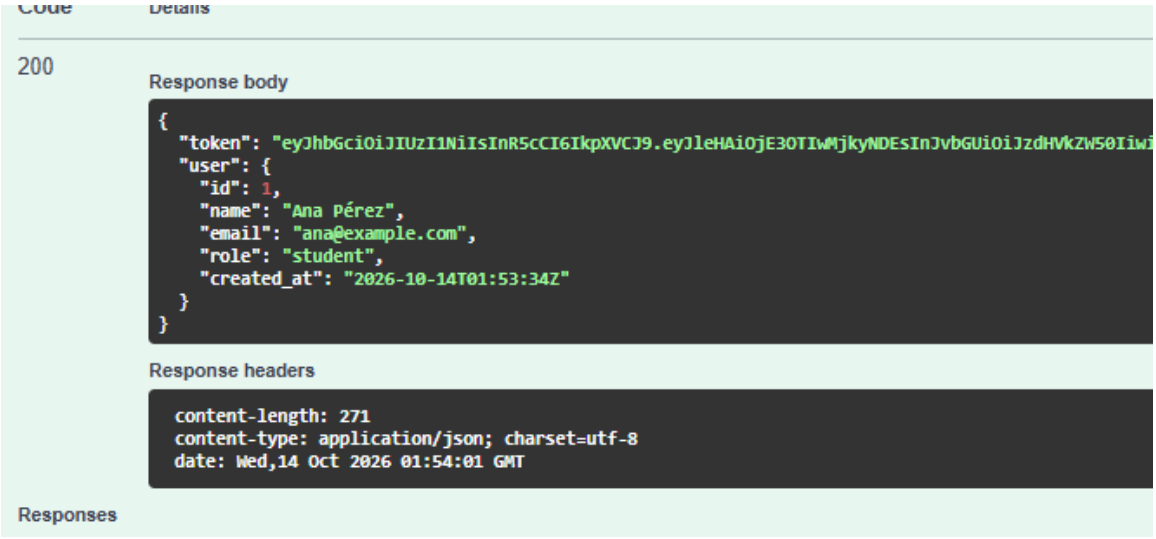
Se configuró el cuerpo de la petición (JSON) para crear un nuevo usuario.



2. Inicio de Sesión (POST /auth/login)

Se ejecutó el inicio de sesión para el usuario creado, obteniendo como respuesta (Código 200) el token JWT necesario para las rutas protegidas.





Responses

Code	Description
200	OK Example Value | Model

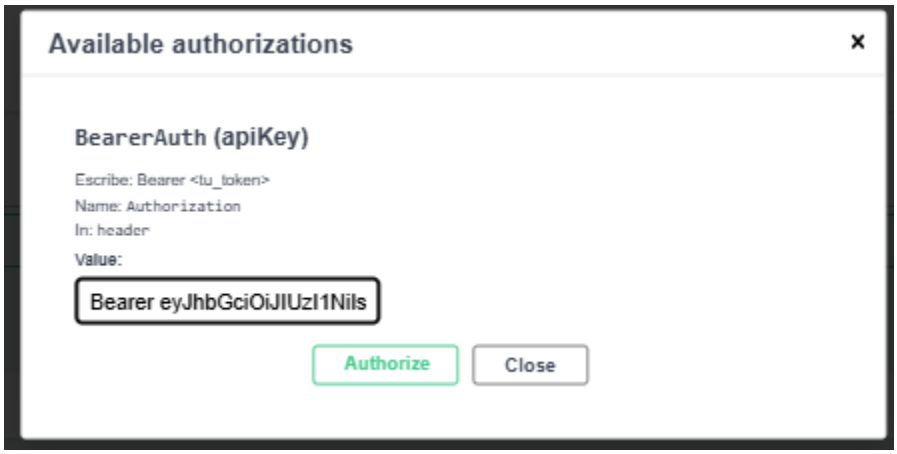
```
{
  "token": "string",
  "user": {
    "created_at": "string",
    "email": "string",
    "id": 0,
    "name": "string",
    "role": "string"
  }
}
```

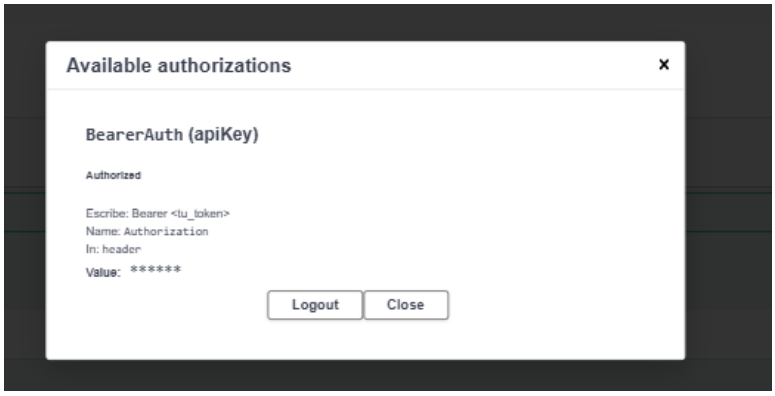
401	Unauthorized Example Value | Model

```
{
  "additionalProp1": "string",
  "additionalProp2": "string",
  "additionalProp3": "string"
}
```

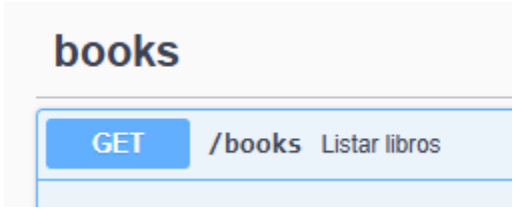
3. Autorización y Consulta de Libros (GET /books)

Se configuró el token en la cabecera Authorization de Swagger (usando el formato Bearer <token>).





Al ejecutar la petición para listar los libros, el servidor devolvió un Código 200 con la información extraída de la base de datos (datos semilla o seed).







Code	Details
200	Response body


```
[
  {
    "id": 1,
    "title": "Cien años de soledad",
    "author": "Gabriel García Márquez",
    "isbn": "9780307474728",
    "category": "Novela",
    "cover_url": "",
    "total_copies": 3,
    "available_copies": 3,
    "created_at": "2026-10-14T01:36:45Z"
  },
  {
    "id": 2,
    "title": "Clean Code",
    "author": "Robert C. Martin",
    "isbn": "9780132350884",
    "category": "Tecnología",
    "cover_url": "",
    "total_copies": 2,
    "available_copies": 2,
    "created_at": "2026-10-14T01:36:45Z"
  },
  {
    "id": 3,
    "title": "El principito",
    "author": "Antoine de Saint-Exupéry",
```


Responses

Code	Description
200	OK

Example Value | Model


```
[
  {
    "author": "string",
    "available_copies": 0,
    "category": "string",
    "cover_url": "string",
    "created_at": "string",
    "id": 0,
    "isbn": "string",
    "title": "string",
    "total_copies": 0
  }
]
```

Preguntas de Comprensión

- 1. ¿Qué diferencia hay entre una imagen y un contenedor? Usa un ejemplo de este proyecto.

Una imagen es una plantilla inmutable de solo lectura que contiene el código, las librerías y las instrucciones necesarias para ejecutar una aplicación. Un contenedor es una instancia en ejecución de esa imagen. En este proyecto, el Dockerfile define cómo construir la imagen de la API en Go. Cuando ejecutamos docker compose up, Docker toma esa imagen y crea un contenedor vivo (llamado biblioteca_api) que ejecuta el binario compilado.

- 2. ¿Qué hace docker compose que no harías con docker run por separado?

docker compose permite orquestar múltiples contenedores (en este caso, la API y MySQL) de forma declarativa mediante un solo archivo YAML. A diferencia de ejecutar múltiples docker run, Compose crea automáticamente una red interna dedicada para que los contenedores se comuniquen entre sí por su nombre de servicio, gestiona el orden de arranque (usando depends_on) y monta los volúmenes de forma centralizada con un solo comando (up).

- 3. ¿Por qué la API encuentra la base de datos con DB_HOST=mysql y no con 127.0.0.1?

Porque dentro de Docker, cada contenedor tiene su propia interfaz de red (su propio localhost o 127.0.0.1). Si la API buscara la base de datos en 127.0.0.1, estaría buscándola dentro de su propio contenedor (donde solo está Go, no MySQL). Al usar DB_HOST=mysql, Docker utiliza su servidor DNS interno para resolver el nombre del servicio (mysql, definido en el compose) a la IP real del contenedor de la base de datos dentro de la red que comparten.

- 4. ¿Qué borra exactamente docker compose down -v y cuándo lo usarías?

Este comando detiene y elimina los contenedores, las redes creadas por el compose y, crucialmente (gracias a la bandera -v), elimina los volúmenes de datos persistentes. En este proyecto, borraría el volumen mysql_data donde MySQL guarda físicamente la información. Lo usaría si quisiera reiniciar la base de datos completamente desde cero, por ejemplo, para que los scripts de migración (migrations/001_init.sql) se vuelvan a ejecutar y recarguen los datos semilla al levantar el proyecto de nuevo.

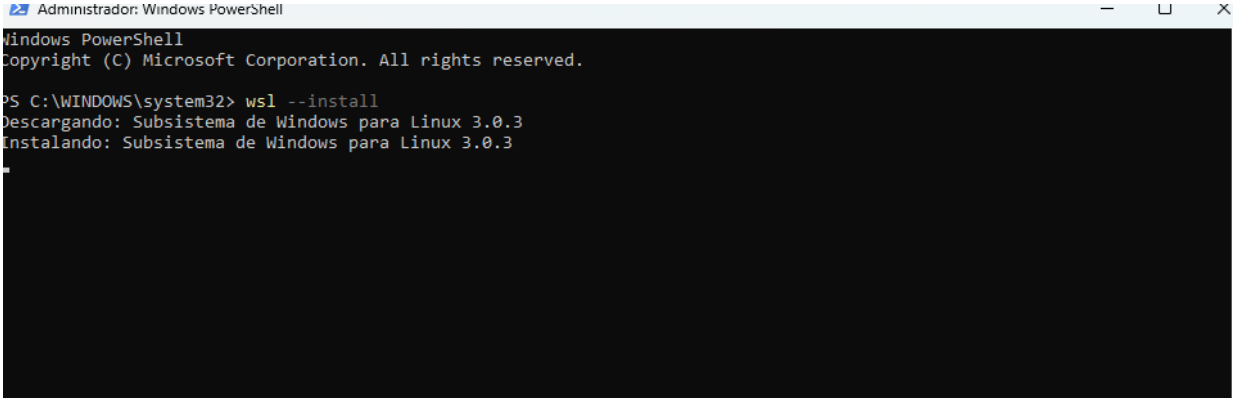
- 5. ¿Qué pasaría si cambias DB_PASSWORD en el .env pero no en el docker-compose.yml? ¿Y al revés?

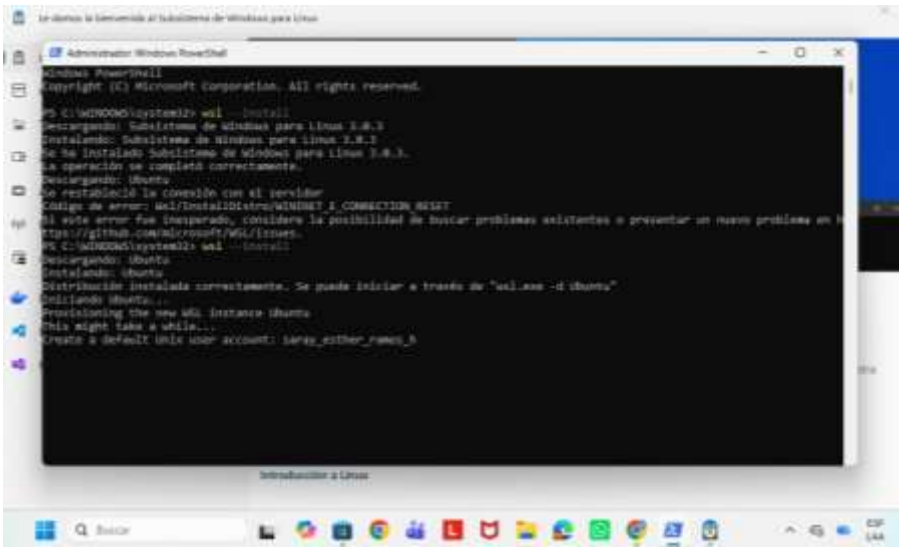
En este proyecto, el archivo docker-compose.yml pasa las variables del .env a los contenedores.
Si el docker-compose.yml está configurado para leer el archivo .env (con la directiva env_file:), cambiar la contraseña en el .env afectará a ambos contenedores (MySQL se inicializará con esa clave y la API la usará para conectarse). Todo funcionaría. Sin embargo, si se cambia la clave "quemada" directamente en la sección environment de MySQL en el docker-compose.yml, pero no se actualiza en el .env, se romperá la conexión: la base de datos se creará con la clave nueva del YAML, pero la API seguirá intentando conectarse con la clave vieja del .env.

Anexos: Problemas Comunes (Solucionados)

Durante la práctica, me encontré con los siguientes escenarios contemplados en el anexo de la guía:

1. 1. Problema con WSL: Al intentar iniciar Docker por primera vez, me apareció un error indicando que WSL no estaba instalado. Lo solucioné ejecutando wsl --install en PowerShell como administrador, configurando mi usuario de Ubuntu y reiniciando la máquina.





2. 2. Error 401 Unauthorized en Swagger ("token no proporcionado"): Al intentar probar el endpoint GET /books, el servidor rechazó la petición. La solución fue modificar el cuadro de autorización de Swagger, asegurándome de escribir la palabra Bearer seguida de un espacio antes de pegar el token JWT.

